

# Introducing two ROS attack variants: breaking one-more unforgeability of BZ blind signatures

Bruno M. F. Ricardo<sup>1</sup>, Lucas C. Cardoso<sup>1</sup> , Leonardo T. Kimura<sup>1</sup>, Paulo S. Barreto<sup>2</sup>, Marcos A. Simplicio Jr<sup>1</sup>

<sup>1</sup> Universidade de São Paulo, São Paulo, Brazil  
{bricardo, lcardoso, lkimura, mjunior}@larc.usp.br

<sup>2</sup> University of Washington, Tacoma, USA  
pbarreto@uw.edu

**Abstract.** In 2023, Barreto and Zanon proposed a three-round Schnorr-like blind signature scheme, leveraging zero-knowledge proofs to produce one-time signatures as an intermediate step of the protocol. The resulting scheme, called BZ, is proven secure in the discrete-logarithm setting under the one-more discrete logarithm assumption with (allegedly) resistance to the Random inhomogeneities in a Overdetermined Solvable system of linear equations modulo a prime number  $p$  attack, commonly referred to as ROS attack. The authors argue that the scheme is resistant against a ROS-based attack by building an adversary whose success depends on extracting the discrete logarithm of the intermediate signing key. In this paper, however, we describe a distinct ROS attack on the BZ scheme, in which a probabilistic polynomial-time attacker can bypass the zero-knowledge proof step to break the one-more unforgeability of the scheme. We also built a BZ variant that, by using one secure hash function instead of two, can prevent this particular attack. Unfortunately, though, we show yet another ROS attack that leverages the BZ scheme’s structure to break the one-more unforgeability principle again, thus revealing that this variant is also vulnerable. These results indicate that, like other Schnorr-based strategies, it is hard to build a secure blind signature scheme using BZ’s underlying structure.

**Keywords:** Blind signature. Schnorr. ROS. Zero-knowledge proofs. Cryptanalysis.

## 1 Introduction

Blind signatures are cryptographic schemes that allow a user to obtain a signature on a message in such a way that the signer does not find out the actual contents of the message. Several applications, including electronic voting [FW24; FOO93] and e-cash [Cha83], build upon the two main security properties provided by these schemes: *perfect blindness* and *one-more unforgeability*. The first means that it should be computationally hard for a signer, after seeing  $n$  message-signature pairs, to correlate any pair to the specific interaction that originated it.

Meanwhile, the second states that the user cannot output  $n + 1$  signatures after having only  $n$  interactions with the signer.

EXISTING BLIND SIGNATURE SCHEMES. There are many blind signature schemes in the literature, each relying on different computational problems and presenting different challenges. Blind RSA [Lys23; DJW23] builds upon the one-more RSA assumption, yielding large signatures and key sizes for modern security levels [Cha83; Cor00]. The blind ECDSA scheme proposed by [QCY21] relies on a combination of additive homomorphic encryption and non-interactive zero-knowledge arguments. Blind BLS and other pairing-based blind signature schemes use pairing-friendly curves, which are slower than standard curves [Bol02; HLW23]. There are also proposals for boosting techniques [KLR21], which can improve the efficiency of some blind signature schemes while ensuring that they remain secure for (polynomially many) concurrent executions of the signing protocol. Finally, post-quantum alternatives are still a work in progress due to performance issues and flaws identified in their corresponding security proofs [Hau+20; Kat+24].

BLIND SCHNORR SIGNATURE. In terms of efficiency, blind signature schemes based on regular Schnorr signatures [Sch90] are among the most prominent constructions [Sch01]. Unfortunately, however, the security of Blind Schnorr signatures commonly relies on the hardness of the Random inhomogeneities in an Overdetermined Solvable system of linear equations (commonly referred to as ROS) problem. Until the early 2000s, this assumption was considered reasonable for practical purposes, since the best-known attack, using a  $k$ -dimensional generalization of the birthday problem, could solve the ROS problem in sub-exponential time [Wag02]. In 2021, though, Benhamouda et al. proposed a polynomial-time algorithm to solve the ROS problem modulo a prime number  $p$  in  $\ell > \log p$  dimensions [Ben+21], showing that malicious users could forge one-more signature using only poly-logarithm concurrent sessions. This led to renewed research interest in the topic, aiming to build Blind Schnorr variants that could resist to ROS attacks while still providing fast computations, small key sizes, and short signatures.

SCHNORR BZ. One of the Schnorr-based blind signature schemes recently proposed to resist ROS attacks, which is the focus of this manuscript, is the so-called *Schnorr Blind Signature Zero-Knowledge* (BZ) scheme, described in [Bar+23; BZ23]. It consists of a three-move protocol where the usual, plain Schnorr signatures are replaced by non-interactive zero-knowledge arguments of knowledge arising from natural properties of Schnorr-based protocols. The security of BZ relies on the *one-more discrete logarithm assumption* [Bel+03], hereby referred to as (OMDL), which is a stronger notion when compared to the discrete logarithm assumption. Further, the authors discuss how to mount possible ROS-based attacks against BZ [BZ23, Supplementary Material A], showing that they fail unless the attacker can solve a discrete logarithm. From those observations, the authors claim that the resulting BZ construction should be resistant against ROS-like attacks.

OUR CONTRIBUTION. In this paper, our contribution is threefold:

- i) We describe a novel ROS-based attack that, albeit possibly of independent interest, is capable of violating the *one-more unforgeability* of the BZ scheme, bypassing the zero-knowledge step of the protocol;
- ii) In an attempt to solve the issue, we present a variant of the BZ scheme, hereby referred to as *modified-BZ* (or *m-BZ* for short), which indeed resists to our first attack;
- iii) Going further, we show yet another variant of ROS-based attack on *m-BZ* Scheme, which once again breaks its *one-more unforgeability* property, showing that it is hard to build a secure blind signature scheme using BZ's underlying structure.

PAPER OUTLINE. The rest of this paper is organized as follows. Section 2 introduces the mathematical notation and color scheme used throughout the paper. Section 3 outlines the required background on Schnorr signatures, the ROS problem, and the BZ scheme. Section 4 describes our first ROS-based attack, as well as the *m-BZ* variant built to avoid this attack. Section 5 presents the second ROS-based attack on the modified BZ scheme, for which we could not find a fix. Section 6 concludes the paper.

## 2 Preliminaries and Notation

SETS. For any finite set  $S$ ,  $x \leftarrow \$ S$  denotes the uniformly random sampling of an element  $x \in S$ , while  $|S|$  is the number of elements in the set. We use  $[n]$  to mean the integers from 1 to  $n$ , and  $[m : n]$  to represent the integers from  $m$  to  $n \geq m$ . We write  $\{0, 1\}^\infty$  to denote the set of all binary strings, and  $\{0, 1\}^*$  for the set of all binary strings with arbitrary finite size. Bit values are represented as  $b \in \{0, 1\}$ .

ALGEBRA. We denote the ring of integers modulo  $p$  by  $\mathbb{Z}_p$ , while its subset of strictly positive values is denoted  $\mathbb{Z}_p^* := \mathbb{Z}_p \setminus \{0\}$ . All arithmetic operations between elements of  $\mathbb{Z}_p$  are done modulo  $p$ , so we omit the  $(\text{mod } p)$  notation for conciseness. We use additive notation for all operations in an Abelian group  $\mathbb{G} = \langle G \rangle$  with prime order  $p$  (i.e.,  $|\mathbb{G}| = p$ ), where  $G$  is a generator of  $\mathbb{G}$ . Group elements are represented with uppercase letters (e.g.,  $G \in \mathbb{G}$ ), while non-group elements are represented as lowercase letters (e.g.,  $g \notin \mathbb{G}$ ). We use bold font style to represent vectors, such as  $\mathbf{v} = (v_1, \dots, v_n) \in \mathbb{Z}_p^n$ . Finally, we also use  $\langle \mathbf{u}, \mathbf{v} \rangle$  to denote the usual inner-product between vectors  $\mathbf{u}, \mathbf{v} \in \mathbb{Z}_p^n$ , i.e.,

$$\langle \mathbf{u}, \mathbf{v} \rangle = \sum_{1 \leq i \leq n} u_i v_i.$$

BLIND SIGNATURES. As further detailed in Section 3.1, the two main actors interacting in a blind protocol are a user  $\mathbf{U}$  and a signer  $\mathbf{S}$ . When describing their actions, we use a sub-index (generally 1 or 2) to represent which step they are executing during the protocol. For example, the first algorithm executed by the signer during the Sch protocol is referred to as  $\text{Sch.S}_1(\text{param})$  where  $\text{param}$  is a set of inputs.

COLOR SCHEME. For easier reference, we use the same color scheme of [BZ23]: key pairs are colored in blue; generators in red; primary commitments in green;

secondary commitments in magenta; primary challenges in purple; secondary challenges in brown; primary answers in turquoise; and secondary answers in wine-red. Transcript components from protocol sessions are indicated with a bar (e.g.,  $\bar{c}$ ), and blinding factors are indicated with Greek letters (except  $\sigma$ , which represents a signature).

### 3 Background

In this section, we present the theoretical background upon which the manuscript is built. We start by presenting Schnorr signatures in both original and blind versions; then, we present the ROS problem and the polynomial-time attack proposed by [Ben+21] to solve it. Lastly, we describe the scheme proposed by Barreto and Zanon [BZ23], highlighting some aspects of its structure and cryptographic assumptions upon which its security depends.

#### 3.1 Schnorr Signature Variants

**ORIGINAL SCHNORR SIGNATURE.** The original Schnorr signature scheme [Sch90] is widely used in modern cryptographic solutions. Examples include cryptocurrencies (e.g., Bitcoin recently adopted Schnorr signatures for curve secp256k1 [WNR20] and Taproot [WNT20]), threshold signatures [Bel+22], identity-based schemes suited for resource-constrained environments [GG09], implicit certification protocols for vehicular networks [Bar+20], among others. The security of Schnorr signatures relies on the hardness of the discrete logarithm in prime order groups  $\mathbb{G}$  such that  $|\mathbb{G}| = p$  and on the strength of the underlying hash function  $\mathcal{H} : \mathbb{G} \times \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$ .

In the Schnorr construction, a signer uses its private key  $x$  to produce a signature  $\sigma$  over (the hash of) a message  $m$  by: (i) sampling a random element  $u$  from  $\mathbb{Z}_p^*$ ; (ii) computing the commitment  $U := uG$ ; (iii) calculating the digest  $c := \mathcal{H}(U, m)$ ; and finally (iv) computing  $z := u + cx \pmod{p}$ . The output of the signature process is  $\sigma = (z, c) \in \mathbb{Z}_p^* \times \mathbb{Z}_p^*$ . The verifier uses the signer's public key  $X := xG$  and the message  $m$  to verify if  $\sigma$  is a valid signature. This is done by: (i) parsing  $\sigma$  as  $(z, c)$ ; (ii) computing  $U' := zG - cX$ ; and then (iii) outputting the boolean value of  $\mathcal{H}(U', m) \stackrel{?}{=} c$ . The challenge  $c$  is typically replaced by  $U$  in the signature. In this case,  $\sigma := (z, U)$ , and the verification process consists in parsing  $\sigma$  as  $(z, U) \in \mathbb{Z}_p^* \times \mathbb{G}^*$  and evaluating the boolean value of  $zG \stackrel{?}{=} U + cX, c := \mathcal{H}(U, m)$ .

**SCHNORR BLIND SIGNATURES.** The Schnorr blind signature scheme [Sch01] derives from the plain scheme: essentially, it takes advantage of the latter's linearity to build a three-move protocol, as illustrated in Figure 1. The procedure is as follows: (i) the signer  $S$  uses the private key  $x$  to produce a primary commitment  $\bar{U} := uG$  along with a state variable  $st_S := (x, u)$ , and sends  $\bar{U}$  to the user  $U$ ; (ii)  $U$  blinds  $\bar{U}$  by combining two randomly sampled blinding factors, named  $\alpha, \beta \leftarrow_{\$} (\mathbb{Z}_p^*)^2$ , respectively with  $G$  and  $X$ , thus obtaining  $U := \bar{U} + \alpha G + \beta X$ ;

(iii)  $U$  also computes  $c := \mathcal{H}(U, m)$  and uses  $\beta$  to calculate  $\bar{c} := c + \beta \bmod p$ , saving the state variable  $st_U := (X, c, U, \alpha, \beta)$  and sending  $\bar{c}$  back to  $S$ ; (iv)  $S$  uses  $st_S$  and  $\bar{c}$  as inputs to produce  $\bar{z} := u + \bar{c}x \bmod p$ , and send it back to  $U$ ; (v) lastly, the user produces the signature  $\sigma = (z, c, U)$ ,  $z := \bar{z} + \alpha \bmod p$ .

As one may notice, the verification process consists in returning the boolean value of  $zG \stackrel{?}{=} U + cX$ ,  $c := \mathcal{H}(U, m)$ , which is done identically to the original signature:

$$\begin{aligned} zG &= \bar{z}G + \alpha G \Rightarrow \\ &= \bar{U} + \bar{c}X + \alpha G = U - \alpha G - \beta X + cX + \beta X + \alpha G \Rightarrow \\ &= U + cX \end{aligned}$$

The security of the Schnorr blind signature scheme relies on the intractability of the ROS problem, which is detailed in the following section.

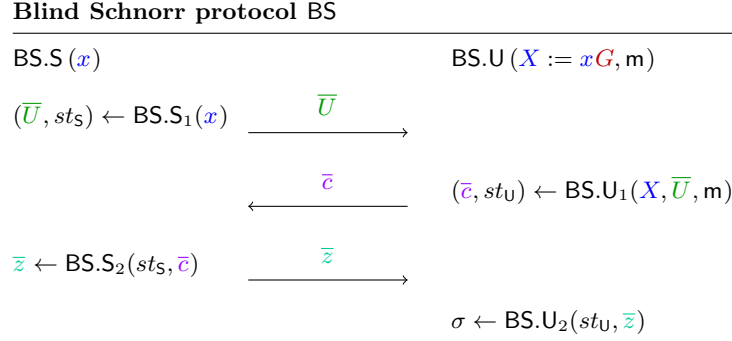


Fig. 1: Three-move blind signature protocol derived from Schnorr signatures scheme.

### 3.2 ROS Problem

The *Random inhomogeneities in a Overdetermined Solvable system of linear equations* problem is broadly studied in [Sch01]. This is done as part of the security assumptions required to overcome the susceptibility to attacks against the one-more unforgeability property of the Schnorr blind signature scheme. The  $\ell$ -dimension ROS problem is stated as follows<sup>3</sup>:

**Definition 1** ([Ben+21]). *Given a prime  $p$ , dimension  $\ell$  and access to a random oracle  $H_{\text{ros}} : \mathbb{Z}_p^\ell \rightarrow \mathbb{Z}_p$ , find  $\ell + 1$  vectors  $\mathbf{v}_i, i \in [\ell + 1]$ , and one vector  $\mathbf{c}$ :*

$$H_{\text{ros}}(\mathbf{v}_i) = \langle \mathbf{c}, \mathbf{v}_i \rangle, \mathbf{v}_i, \mathbf{c} \in \mathbb{Z}_p^\ell, i \in [\ell + 1]$$

<sup>3</sup> Note that, in Definition 1, we use our own notation applied to an alternative formulation of ROS from [HKL19, Fig. 3], where linear functions are used instead.

The first positive result presenting a solution with sub-exponential time complexity to the ROS problem was due to Wagner [Wag02]. This was a consequence of his work about the *Generalized Birthday Paradox to  $\ell$  dimensions*, defined as:

**Definition 2.** *Let  $i \in [\ell]$  such that  $\mathfrak{L}_i$  corresponds to a list of random elements. Find  $x_i \in \mathfrak{L}_i$ ,  $i \in [\ell]$  such that  $x_1 + \dots + x_\ell \equiv 0 \pmod{p}$ .*

Wagner’s algorithm solved the problem in  $O(\ell \cdot 2^{\lceil \log p \rceil / (1 + \lceil \log p \rceil)})$  time complexity. At the time, it had major consequences to cryptanalysis, especially to schemes using the hardness of the ROS problem as part of their security assumptions. Later, Benhemouda et al. expanded Wagner’s approach and proposed a polynomial-time algorithm to solve the ROS problem [Ben+21], thus proving the insecurity of many Schnorr-based blind signature schemes. One of their main results is stated as follows:

**Theorem 1** ([Ben+21]). *Let  $\text{PGen}(1^\lambda)$  be a parameter generation algorithm that, given as input the security parameter  $\lambda$  in unary, outputs an odd prime of (bit) length  $\lambda = \lceil \log p \rceil$ . If  $\ell \geq \lambda$ , then there exists an adversary that runs in polynomial time and solves the ROS problem relative to  $\text{PGen}$  with dimension  $\ell$ .*

The idea of the proof is to build an attacker capable of constructing a polynomial whose coefficients depend on the information extracted from  $\ell$  opened sessions. Then, it leverages the recursive nature of the ROS problem to obtain the target values  $\mathbf{c}$  and  $\mathbf{v}_{\ell+1}$  from Definition 1, since the index  $i$  lies in the interval  $[\ell + 1]$ . It is worth mentioning that an attack combining Wagner’s  $k$ -list algorithm and Theorem 1 is also addressed in [Ben+21] when discussing attacks for  $\ell < \log p$ . We refer the reader to [Ben+21] for a detailed discussion.

### 3.3 Barreto and Zanon Blind signature Scheme

Seeking to produce a three-move Schnorr-based blind signature scheme resilient to ROS attacks, Barreto and Zanon [BZ23] adapted the scheme by providing a *zero-knowledge proof of knowledge* (ZKPoK) of the  $z$  component, instead of providing its actual value in the blind signature. More precisely, the scheme uses the  $z$  part of a Schnorr signature  $\sigma = (z, U)$  as a private key, and then produces a second signature that can be used as a proof. This technique is similar to the one used in [GG09], in the context of identity-based scenarios, and formally proved a *succinct non-interactive argument of knowledge* (SNARK) by Barreto et al. [BJZ22]. The resulting blind signature scheme is called BZ, and its three-move protocol is shown in Figure 2.

The BZ scheme is built on a secure abelian group  $\mathbb{G}$  of prime order  $p \approx 2\lambda$  generated by  $G = \langle \mathbb{G} \rangle$ , and two distinct hash functions defined as  $\mathcal{H} : \mathbb{G} \times \{0, 1\}^* \mapsto \mathbb{Z}_p^*$  and  $\mathcal{G} : \mathbb{G} \mapsto \mathbb{Z}_p^*$ . It leverages the signing algorithm of BS to provide a *primary* answer  $z$  to a *primary* challenge  $c$  and *primary* commitment  $U$ . Then, instead of returning  $z$ , it uses this value as a private key and returns a proof of knowledge consisting of a *secondary* answer  $w$  to a *secondary* challenge  $d$  and *secondary* commitment  $V$ . It is worth mentioning that the second process is the

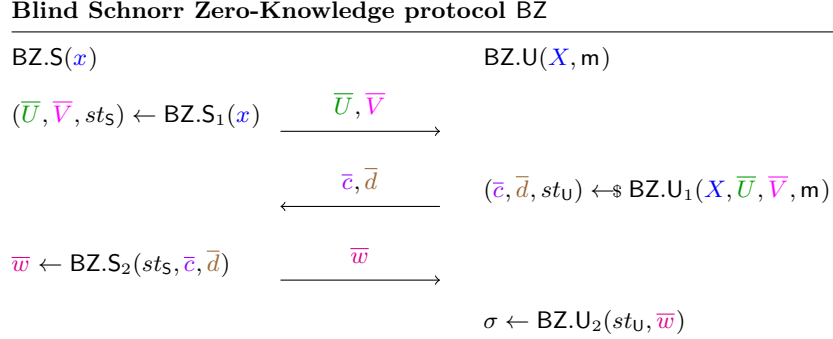


Fig. 2: Three-move blind signature scheme proposed by Barreto and Zanon.

same as in the primary challenge, but with different values. Figure 3 presents the complete specification of the BZ blind signature scheme. Compared to the original description from [BZ23], the only visual difference is that we do not explicitly show  $z$  and  $h$ , but use their values directly for conciseness.

The security of the BZ scheme is based on the *one-more discrete-logarithm assumption* (OMDL), stated as follows:

**Definition 3 (One-more Discrete Logarithm Assumption).** *Let  $\mathbb{G}$  be an abelian group of prime order  $p$  generated by  $G$  and  $\mathcal{A}$  a probabilistic polynomial-time algorithm. Let  $\mathcal{O}_{\text{DL}}$  be an oracle that returns the discrete logarithm of any group element submitted by the adversary  $\mathcal{A}$ . Upon receiving the challenges  $X_i$ ,  $i \in [\ell]$ , it is hard for  $\mathcal{A}$  to extract the discrete logarithm of all  $X_i$  and the one-more discrete logarithm  $x_{\ell+1} := \log_G X_{\ell+1}$ .*

The idea behind the security proof presented in [BZ23] is to build a reduction that breaks the *impersonation under concurrent attacks* (IMP-CA) of the *one-more unforgeability assumption*. At some point, the attacker must extract the discrete logarithm  $z$ , i.e., a zero-knowledge proof of knowledge of the primary signature. The proof appears to have some gaps, though, as noticed by [CTZ24]: their main criticism refers to the obtained concurrent security since a Schnorr identification scheme that is IMP-CA secure does not yield a secure blind signature. Despite those concerns in [CTZ24], to our knowledge no practical attack against BZ has been proposed in the literature. On the other hand, [BZ23] does show how to instantiate variants of ROS-based attacks against the scheme, and why they fail.

## 4 Attacking BZ

In this section, we propose a new ROS-based attack against BZ, and show its correctness. Subsequently, we show a possible fix to the original BZ scheme, producing *m-BZ* – a *modified* version of the original scheme that uses one

|                                                                                                   |                                                                                                   |
|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>BZ.PG(<math>1^\lambda</math>)</b>                                                              | <b>BZ.S<sub>2</sub>(st<sub>S</sub>, <math>\bar{c}, \bar{d}</math>)</b>                            |
| 1 : <b>return</b> par := ( $p, \mathbb{G}, G, \mathcal{H}, \mathcal{G}$ )                         | 20 : ( $x, \text{par}, u, v$ ) $\leftarrow$ st <sub>S</sub>                                       |
| <b>BZ.KG(par)</b>                                                                                 | 21 : ( $p, \mathbb{G}, G, \mathcal{H}, \mathcal{G}$ ) $\leftarrow$ par                            |
| 2 : ( $p, \mathbb{G}, G, \mathcal{H}, \mathcal{G}$ ) $\leftarrow$ par                             | 22 : $\bar{z} \leftarrow u - \bar{c}x$                                                            |
| 3 : $x \leftarrow \mathbb{Z}_p^*, X \leftarrow xG$                                                | 23 : $\bar{w} \leftarrow v - \bar{d}(\boxed{u - \bar{c}x})$                                       |
| 4 : sk $\leftarrow$ ( $x, \text{par}$ ), pk $\leftarrow$ ( $X, \text{par}$ )                      | 24 : <b>return</b> $\bar{w}$                                                                      |
| 5 : <b>return</b> (sk, pk)                                                                        | <b>BZ.U<sub>2</sub>(st<sub>U</sub>, <math>\bar{w}</math>)</b>                                     |
| <b>BZ.S<sub>1</sub>(sk)</b>                                                                       | 25 : ( $X, \text{par}, c, d, U, V, \pi, \delta, \rho, \varepsilon$ ) $\leftarrow$ st <sub>U</sub> |
| 6 : ( $x, \text{par}$ ) $\leftarrow$ sk                                                           | 26 : ( $p, \mathbb{G}, G, \mathcal{H}, \mathcal{G}$ ) $\leftarrow$ par                            |
| 7 : ( $p, \mathbb{G}, G, \mathcal{H}, \mathcal{G}$ ) $\leftarrow$ par                             | 27 : <b>if</b> $\bar{V} \neq \bar{w}G + \bar{d}\bar{U} - \bar{c}\bar{d}X$ :                       |
| 8 : $u \leftarrow \mathbb{Z}_p^*, v \leftarrow \mathbb{Z}_p^*$                                    | 28 : <b>return</b> $\perp$                                                                        |
| 9 : $\bar{U} \leftarrow uG, \bar{V} \leftarrow vG$                                                | 29 : $w \leftarrow \rho\pi\bar{w} - d\delta + \varepsilon$                                        |
| 10 : st <sub>S</sub> $\leftarrow$ ( $x, \text{par}, u, v$ )                                       | 30 : <b>return</b> $\sigma := (w, c, d, U, V)$                                                    |
| 11 : <b>return</b> ( $\bar{U}, \bar{V}, \text{st}_S$ )                                            | <b>BZ.Ver(pk, m, <math>\sigma</math>)</b>                                                         |
| <b>BZ.U<sub>1</sub>(pk, m)</b>                                                                    | 31 : ( $X, \text{par}$ ) $\leftarrow$ pk                                                          |
| 12 : ( $X, \text{par}$ ) $\leftarrow$ pk                                                          | 32 : ( $p, \mathbb{G}, G, \mathcal{H}, \mathcal{G}$ ) $\leftarrow$ par                            |
| 13 : ( $p, \mathbb{G}, G, \mathcal{H}, \mathcal{G}$ ) $\leftarrow$ par                            | 33 : ( $w, c, d, U, V$ ) $\leftarrow$ $\sigma$                                                    |
| 14 : ( $\pi, \delta, \rho, \varepsilon$ ) $\leftarrow \mathbb{Z}_p^*$                             | 34 : <b>if</b> $c \neq \mathcal{H}(U, m) \cup d \neq \mathcal{G}(V)$ :                            |
| 15 : $U \leftarrow \pi\bar{U} + \delta G, V \leftarrow \rho\pi\bar{V} + \varepsilon G$            | 35 : <b>return</b> 0                                                                              |
| 16 : $c \leftarrow \mathcal{H}(U, m), d \leftarrow \mathcal{G}(V)$                                | 36 : <b>return</b> $V \stackrel{?}{=} wG + dU - cdX$                                              |
| 17 : $\bar{c} \leftarrow c/\pi, \bar{d} \leftarrow d/\rho$                                        |                                                                                                   |
| 18 : st <sub>U</sub> $\leftarrow$ ( $X, \text{par}, c, d, U, V, \pi, \delta, \rho, \varepsilon$ ) |                                                                                                   |
| 19 : <b>return</b> ( $\bar{c}, \bar{d}, \text{st}_U$ )                                            |                                                                                                   |

Fig. 3: The complete BZ scheme. We use  $\boxed{\text{boxes}}$  to highlight the definition of  $\bar{z}$  in line 22 and its usage in line 23 of BZ.S<sub>2</sub>(st<sub>S</sub>,  $\bar{c}, \bar{d}$ ).

(instead of two) hash functions to produce a blind signature. Our attack uses an strategy similar to the one discussed in [Ben+21], i.e., we make use of information extracted from  $\ell$  concurrent sessions to build a probabilistic polynomial-time adversary  $\mathcal{A}$  able to forge the  $(\ell + 1)$ -th blind signature on some chosen message. **ATTACK DESCRIPTION.** The attacker  $\mathcal{A}$  starts by opening  $\ell > \log p$  sessions with the server to receive the commitments  $\bar{U}_i$  and  $\bar{V}_i$ ,  $i \in [\ell]$ . The attacker then randomly samples  $\pi_i, \rho_i, \varepsilon_i$  from  $\mathbb{Z}_p^*$  and computes  $V_i$  as follows:

$$V_i := \rho_i \pi_i \bar{V}_i + \varepsilon_i G \quad (1)$$



The *secondary* challenges are calculated according to the specification:  $d_i := \mathcal{G}(V_i)$  and  $\bar{d}_i := d_i/\rho_i$ . Now, the adversary needs to generate two different values for the primary challenges  $c_i$ , as in [Ben+21]. For that, it has to also use two different  $U_i$ , since  $c_i = \mathcal{H}(U_i, \mathbf{m})$ . Hence, for each bit  $b \in \{0, 1\}$ ,  $\mathcal{A}$  generates different  $\delta_{i,b}$  randomly sampled over  $\mathbb{Z}_p^*$  and calculates  $U_{i,b}$  as follows:

$$U_{i,b} := \pi_i \bar{U}_i + \delta_{i,b} G \quad (2)$$

$\mathcal{A}$  then computes the *primary* challenges  $c_{i,b} := \mathcal{H}(U_{i,b}, \mathbf{m})$  followed by  $\bar{c}_{i,b} := c_{i,b}/\pi_i$ , as in the original scheme. The adversary defines  $\mathbf{b} \in \{0, 1\}^\ell$  as the chosen vector of bit challenges to receive the answers  $\bar{w}_i$ ,  $i \in [\ell]$ , satisfying:

$$\bar{w}_i G + \bar{d}_i \bar{U}_i - \bar{c}_{i,b_i} \bar{d}_i X = \bar{V}_i, \quad i \in [\ell]$$

For each  $i \in [\ell]$ ,  $\mathcal{A}$  can iterate the equation  $p_i$  times to obtain a linear combination of equations, that is:

$$\sum_{1 \leq i \leq \ell} p_i (\bar{w}_i G + \bar{d}_i \bar{U}_i - \bar{c}_{i,b_i} \bar{d}_i X) = \sum_{1 \leq i \leq \ell} p_i \bar{V}_i$$

With a slight abuse of notation, this can be rewritten as the following inner product equation by leveraging the fact that the considered group  $\mathbb{G}$  is abelian and written additively:

$$\langle \mathbf{p}, \bar{\mathbf{w}} \rangle G + \langle \mathbf{p}, \bar{\mathbf{d}} \bar{\mathbf{U}} \rangle - \langle \mathbf{p}, \bar{\mathbf{c}} \bar{\mathbf{d}} \rangle X = \langle \mathbf{p}, \bar{\mathbf{V}} \rangle \quad (3)$$

ATTACK RATIONALE. The essence of the attack consists in manipulating  $\langle \mathbf{p}, \bar{\mathbf{c}}_{i,b_i} \bar{\mathbf{d}}_i \rangle$ , i.e., choosing  $\mathbf{p}$  in such a way that, for any  $y \in \mathbb{Z}_p^*$ , the value of  $\langle \mathbf{p}, \bar{\mathbf{c}}_{i,b_i} \bar{\mathbf{d}}_i \rangle$  will be equal to  $y$  when evaluated on specific values  $(b_1, \dots, b_\ell) =: \mathbf{b} \in \{0, 1\}^\ell$ . To succeed, the attacker must find  $\mathbf{p} \in \mathbb{Z}_p^\ell$  such that, for all  $y \in \mathbb{Z}_p^*$ , there is a  $\mathbf{x} := (\bar{c}_{1,b_1} \bar{d}_1, \dots, \bar{c}_{\ell,b_\ell} \bar{d}_\ell) \in \mathbb{Z}_p^\ell$  satisfying  $\langle \mathbf{p}, \mathbf{x} \rangle = y$ .  $\mathcal{A}$  can find  $\mathbf{p}$  by defining the polynomial  $\mathcal{P}(\mathbf{x})$  as follows:

$$\mathcal{P}(\mathbf{x}) := \sum_{1 \leq i \leq \ell} 2^{i-1} \frac{(x_i - \bar{c}_{i,0} \bar{d}_i)}{\bar{c}_{i,1} \bar{d}_i - \bar{c}_{i,0} \bar{d}_i}$$

Note that  $\bar{c}_{i,0} = \bar{c}_{i,1}$  with negligible probability in  $\lambda$ . We can then rewrite this equation in terms of  $\langle \mathbf{p}, \mathbf{x} \rangle$  and an additional factor  $p_0$  by separating the numerator of  $\mathcal{P}(\mathbf{x})$ :

$$\mathcal{P}(\mathbf{x}) = \sum_{1 \leq i \leq \ell} p_i x_i + p_0 = \langle \mathbf{p}, \mathbf{x} \rangle + p_0 \quad (4)$$

where

$$p_i := \frac{2^{i-1}}{\bar{c}_{i,1} \bar{d}_i - \bar{c}_{i,0} \bar{d}_i}, p_0 := \sum_{1 \leq i \leq \ell} 2^{i-1} \frac{-\bar{c}_{i,0} \bar{d}_i}{\bar{c}_{i,1} \bar{d}_i - \bar{c}_{i,0} \bar{d}_i} \quad (5)$$

With this construction,  $\mathcal{P}(\bar{c}_{1,b_1}\bar{d}_1, \dots, \bar{c}_{\ell,b_\ell}\bar{d}_\ell) = \sum_{i=1}^{\ell} 2^{i-1}b_i$ , i.e.,  $\mathcal{P}(\bar{c}_{1,b_1}\bar{d}_1, \dots, \bar{c}_{\ell,b_\ell}\bar{d}_\ell)$  is equal to the binary representation of  $\mathbf{b} \in \{0, 1\}^\ell$ . In order to find  $\mathbf{x}$  such that  $\langle \mathbf{p}, \mathbf{x} \rangle = y$ , the attacker picks  $\mathbf{b}$  as the binary representation of  $y + p_0$ , and sets  $\mathbf{x} = (\bar{c}_{1,b_1}\bar{d}_1, \dots, \bar{c}_{\ell,b_\ell}\bar{d}_\ell)$ . This works because:

$$\begin{aligned} \langle \mathbf{p}, \bar{c}_{i,b_i}\bar{d}_i \rangle &= \mathcal{P}(\bar{c}_{1,b_1}\bar{d}_1, \dots, \bar{c}_{\ell,b_\ell}\bar{d}_\ell) - p_0 \\ &= (y + p_0) - p_0 \\ &= y \end{aligned}$$

After this,  $\bar{c}_{i,b_i}$ ,  $\bar{d}_i$ ,  $i \in [\ell]$  and  $\mathbf{p}$  are well defined. Now,  $\mathcal{A}$  uses Equation 3 to produce the commitments and challenges of the forged signature. For this purpose,  $\mathcal{A}$  computes the parameters in the specified order:

$$\begin{cases} V^* := \langle \mathbf{p}, \bar{\mathbf{V}} \rangle \\ d^* := \mathcal{G}(V^*) \\ U^* := \frac{\langle \mathbf{p}, \bar{d}_i \bar{U}_i \rangle}{d^*} \\ c^* := \mathcal{H}(U^*, \mathbf{m}^*) \end{cases} \quad (6)$$

Finally, the attacker chooses  $\mathbf{b}$  as the binary representation of  $c^*d^* + p_0$  such that  $\langle \mathbf{p}, \bar{c}_{i,b_i}\bar{d}_i \rangle = c^*d^*$  and sends the challenges accordingly (i.e., sets  $\delta_i = \delta_{i,b_i}$ ,  $U_i = U_{i,b_i}$ ,  $c_i = c_{i,b_i}$ ,  $\bar{c}_i = \bar{c}_{i,b_i}$  and sends  $\bar{c}_i, \bar{d}_i$ ). After receiving answers  $\bar{\mathbf{w}} = (\bar{w}_1, \dots, \bar{w}_\ell)$ , the adversary can output the legitimate signatures:

$$\sigma_i = (\rho_i \pi_i \bar{w}_i - \delta_i d_i + \epsilon, c_i, d_i, U_i, V_i), \quad \text{for } i \in [\ell]$$

Those are known to be valid due to the correctness of the BZ. Finally,  $\mathcal{A}$  obtains the forged signature by calculating  $w^* := \langle \mathbf{p}, \bar{\mathbf{w}} \rangle$  and outputting  $\sigma^* = (w^*, c^*, d^*, U^*, V^*)$ . Its validity can be verified from the definitions listed in (6):

$$\begin{aligned} V^* = \langle \mathbf{p}, \bar{\mathbf{V}} \rangle &\Rightarrow \\ &= \langle \mathbf{p}, \bar{\mathbf{w}} \rangle G + \langle \mathbf{p}, \bar{d}_i \bar{U}_i \rangle - \langle \mathbf{p}, \bar{c}_{i,b_i} \bar{d}_i \rangle X \stackrel{(6)}{\Rightarrow} \\ &= w^* G + d^* U^* - c^* d^* X \end{aligned}$$

This is the verification equation for the forged signature  $\sigma^*$ , which corresponds to a valid signature on message  $\mathbf{m}^* = \mathbf{m}_{\ell+1}$   $\square$ .

EXPERIMENTAL RESULTS. The results of Table 1 show the average time in seconds to break one-more unforgeability of BZ instantiated over different secure elliptic curves [Ran23]. Our experimental setup consists of common off-the-shelf equipment (an Intel Core i3-4005U @ 1.70GHz with 4GB of RAM) to simulate the attack using SageMath.

| Elliptic Curve | Expected security level | Time (s) |
|----------------|-------------------------|----------|
| P-224          | 112                     | 5.4      |
| P-256          | 128                     | 6.5      |
| P-384          | 192                     | 16.6     |
| P-521          | 256                     | 43.3     |

Table 1: Time measurements of attack on BZ instantiated over different secure elliptic curves.

#### 4.1 Suggested fix: designing the $m$ -BZ scheme

One possible approach to neutralize the described ROS attack consists in using a single secure hash function, introducing a dependence between the challenges. In this case, both challenges  $c$  and  $d$  would be obtained simultaneously, i.e.,  $(c, d) := \mathcal{H}(U, V, m)$ ,  $\mathcal{H} : \mathbb{G}^2 \times \{0, 1\}^* \mapsto \mathbb{Z}_p^*$ . Line 16 of algorithm BZ.U<sub>1</sub>(pk, m) would be replaced by  $(c, d) := \mathcal{H}(U, V, m)$  after the primary and secondary commitments are calculated:  $U := \pi\bar{U} + \delta G$  and  $V := \rho\pi\bar{V} + \varepsilon G$ .

This modification renders the described attack impossible because it creates a circularity between  $d^*$ ,  $U^*$  and  $c^*$  in Equation 6. Consequently, the attacker would be unable to perform the first steps of the attack, which involve calculating a *secondary* challenge  $\bar{d}_i$  followed by the generation of two different *primary* challenges  $\bar{c}_{i,b_i}$ ,  $b_i \in \{0, 1\}$ .

### 5 Attack on $m$ -BZ

Although the  $m$ -BZ construction prevents the attack described in Section 4, unfortunately it is still vulnerable to a second probabilistic polynomial-time attack that breaks its one-more unforgeability. The core of the attack consists in solving the problem of generating blinded challenges  $c_b$  with the same  $d$ . Essentially, this means that, given a message  $m$  and commitments  $\bar{U}$  and  $\bar{V}$ , the attacker  $\mathcal{A}$  must generate *only* one  $\bar{d}$  and two possible  $\bar{c}_b$ ,  $b \in \{0, 1\}$ . If  $\mathcal{A}$  sends the challenges  $(\bar{c}_b, \bar{d})$  to the signer and receives an answer  $\bar{w}_b$ ,  $\mathcal{A}$  can compute a signature  $\sigma_b := (w_b, c_b, d_b, U_b, V_b)$ .

**ATTACK'S DESCRIPTION.** The attack works as follows. After sending the challenges,  $\mathcal{A}$  receives an answer  $\bar{w}_b$  satisfying the verification equation:

$$\bar{w}_b G + \bar{d}\bar{U} - \bar{c}_b \bar{d} X = \bar{V}$$

This equation is equivalent to:

$$\bar{w}_b G - \bar{c}_b \bar{d} X = \bar{V} - \bar{d}\bar{U} \quad (7)$$

The attacker sets the secondary commitment to  $V_b = \bar{V} - \bar{d}\bar{U}$ , so the primary commitment  $U_b$  can be chosen freely. In particular, it can be set to a known discrete logarithm, i.e.,  $U_b := u_b G$ ,  $u_b \leftarrow \mathbb{Z}_p^*$ , with  $u_b$  generated by the attacker.  $\mathcal{A}$  generates two random nonces  $u_b$  and sets  $\bar{U}_b = u_b G$ . With the two commitments

defined,  $c_b$  and  $d_b$  are calculated as usual, i.e.,  $(c_b, d_b) := \mathcal{H}(U_b, V_b, \mathbf{m})$ . Then,  $\mathcal{A}$  can manipulate Equation 7 using  $V_b = \bar{V} - \bar{d}\bar{U}$ :

$$\begin{aligned} V_b &= \bar{w}_b G - \bar{c}_b \bar{d} X \Rightarrow \\ &= \bar{w}_b G + (d_b u_b G - d_b u_b G) - \bar{c}_b \bar{d} X \Rightarrow \\ &= (\bar{w}_b - d_b u_b) G + d_b u_b G - \bar{c}_b \bar{d} X \Rightarrow \\ &= \underbrace{(\bar{w}_b - d_b u_b)}_{=: w_b} G + d_b U_b - \bar{c}_b \bar{d} X \end{aligned}$$

Hence, obtaining the equation:

$$V_b = w_b G + d_b U_b - \bar{c}_b \bar{d} X \quad (8)$$

ATTACK RATIONALE. The core of the attack consists in ensuring that Equation 7 holds for signature  $\sigma_b := (w_b, c_b, d_b, U_b, V_b)$ . For that purpose,  $\mathcal{A}$  has to choose  $\bar{c}_b$  and  $\bar{d}$  such that  $\bar{c}_b \bar{d} = c_b d_b$ . By a simple substitution of  $\bar{c}_b \bar{d} = c_b d_b$  in Equation 8, the term  $\bar{c}_b \bar{d} X$  becomes  $c_b d_b X$ , so  $\sigma_b$  is verified as valid. Due to the independence of  $\bar{d}$  with respect to  $b$ ,  $\mathcal{A}$  can set  $\bar{d}$  to be a non-zero constant. This means that  $\mathcal{A}$  can calculate  $\bar{c}_b$  as:

$$\bar{c}_b := \frac{c_b d_b}{\bar{d}} \quad (9)$$

Finally,  $\mathcal{A}$  computes  $w_b := \bar{w}_b - d_b u_b$ , and the valid signature on message  $\mathbf{m}$ :  $\sigma = (w_b, c_b, d_b, U_b, V_b)$ .

Building upon this core procedure, the complete attack is as follows: the adversary opens  $\ell$  concurrent sessions with the signer, receiving commitments  $\bar{U}_i$  and  $\bar{V}_i$  for  $i \in [\ell]$ . Then, for each session,  $\mathcal{A}$  calculates the commits and challenges (blinded or not) as explained above: (i) sets the constant value  $\bar{d}_i \leftarrow \mathbb{Z}_p^*$ ; (ii) uses Equation 7 to obtain  $V_i := \bar{V}_i - \bar{d}_i \bar{U}_i$ ; (iii) makes  $U_{i,b} := u_{i,b} G$ ,  $b \in \{0, 1\}$ ,  $u_{i,b} \leftarrow \mathbb{Z}_p^*$ ; (iv) computes  $(c_{i,b}, d_{i,b}) := \mathcal{H}(U_{i,b}, V_i, \mathbf{m}_i)$ ; and finally (v) uses Equation 9 to calculate  $\bar{c}_{i,b}$ .

After that,  $\mathcal{A}$  computes the coefficients  $p_i$ ,  $i \in [\ell]$ , and the constant term  $p_0$  of the linear polynomial  $\mathcal{P}$  exactly as in the attack described in Section 4, using Equation 5.  $\mathcal{A}$  can then employ the same trick of setting the commit  $U^*$  to a value with a known discrete logarithm: it generates a random  $u^* \leftarrow \mathbb{Z}_p^*$  and calculates the commitments and challenges of the forged signature following Equation 3:

$$\begin{cases} V^* := \langle \mathbf{p}, \bar{\mathbf{V}} \rangle - \langle \mathbf{p}, \bar{d}_i \bar{U}_i \rangle \\ U^* := u^* G \\ (c^*, d^*) := \mathcal{H}(U^*, V^*, \mathbf{m}^*) \end{cases} \quad (10)$$

The attacker chooses once again  $\mathbf{b} \in \{0, 1\}^\ell$  as the binary representation of  $c^* d^* + p_0$  and sends the challenges accordingly (i.e., sets  $u_i = u_{i,b_i}$ ,  $U_i = U_{i,b_i}$ ,

$c_i = c_{i,b_i}$ ,  $d_i = d_{i,b_i}$ ,  $\bar{c}_i = \bar{c}_{i,b_i}$  and sends  $\bar{c}_i, \bar{d}_i$ ). After receiving answers  $\bar{\mathbf{w}}$ , the adversary can output the legitimate signatures:

$$\sigma_i = (\bar{w}_i - d_i u_i, c_i, d_i, U_i, V_i), \quad \text{for } i \in [\ell]$$

All those signatures are valid for the reasons explained at the beginning of this section. Finally, the attacker can produce the forged signature by calculating  $w^* := \langle \mathbf{p}, \bar{\mathbf{w}} \rangle - d^* u^*$  and outputting  $\sigma^* = (w^*, c^*, d^*, U^*, V^*)$ . Its validity can be verified by following the definitions given in (10):

$$\begin{aligned} V^* &= \langle \mathbf{p}, \bar{\mathbf{V}} \rangle - \langle \mathbf{p}, \bar{d}_i \bar{U}_i \rangle \Rightarrow \\ &= \langle \mathbf{p}, \bar{\mathbf{w}} \rangle G - \langle \mathbf{p}, \bar{c}_{i,b_i} \bar{d}_i \rangle X \Rightarrow \\ &= (w^* + d^* u^*) G - c^* d^* X \Rightarrow \\ &= w^* G + d^* U^* - c^* d^* X \quad \square \end{aligned}$$

**EXPERIMENTAL RESULTS.** Table 2 shows the average time, in seconds, to break one-more unforgeability of  $m$ -BZ blind signature scheme. Our experiments use the same setup and safe elliptic curves as in the practical results of Section 4. We can notice that, while the usage of one hash function in  $m$ -BZ hinders our first ROS-like attack, it also allows forgeries to be obtained in this second attack with 25% less time on average.

| Elliptic Curve | Expected security level | Time (s) |
|----------------|-------------------------|----------|
| P-224          | 112                     | 4.0      |
| P-256          | 128                     | 4.7      |
| P-384          | 192                     | 12.4     |
| P-521          | 256                     | 31.2     |

Table 2: Time measurements of attack on  $m$ -BZ instantiated over different secure elliptic curves.

## 6 Conclusion

In this paper, we show how to successfully break one-more unforgeability of BZ blind signature scheme. This is accomplished by means of a new ROS-like attack capable of circumventing BZ's the zero-knowledge step, designed to be an obstacle to such types of attacks. Further, we propose modifications in the original BZ, leading to the so-called  $m$ -BZ scheme: by using one secure hash function instead of two, we can effectively prevent the described attack. Unfortunately, however,  $m$ -BZ is susceptible to a second attack, as we can build a new probabilistic polynomial-time algorithm to forge a valid signature from  $\ell$  concurrent sessions.

Our experimental results show that the one-more unforgeability property can be broken in seconds using common off-the-shelf equipment, even when BZ and  $m$ -BZ are instantiated with different secure elliptic curves. Both attacks are written in SageMath and uploaded in the following git [repository](#). These results indicate that, like other Schnorr-based strategies, it is hard to build a secure blind signature scheme using BZ’s underlying structure.

## References

- [BZ23] P. Barreto and G. Zanon. *Blind signatures from Zero-knowledge arguments*. Cryptology ePrint Archive, Paper 2023/067. 2023. URL: <https://eprint.iacr.org/2023/067>.
- [Bar+20] P. Barreto et al. “Schnorr-based implicit certification: Improving the security and efficiency of vehicular communications”. In: *IEEE Transactions on Computers* 70.3 (2020), pp. 393–399.
- [Bar+23] P. Barreto et al. “Blind signatures from Zero knowledge in the Kummer variety”. In: *Anais do XXIII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais*. Juiz de Fora/MG: SBC, 2023, pp. 139–152. DOI: [10.5753/sbseg.2023.233503](#).
- [BJZ22] Paulo Barreto, Marcos Simplicio Jr, and Gustavo Zanon. “Succinct Non-interactive Arguments of Knowledge from Supersingular Isogenies”. In: *Anais do XXII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais*. Santa Maria: SBC, 2022, pp. 181–194. DOI: [10.5753/sbseg.2022.225292](#).
- [Bel+03] M. Bellare et al. “The One-More-RSA-Inversion Problems and the Security of Chaum’s Blind Signature Scheme”. In: *Journal of Cryptology* 16.3 (June 2003), pp. 185–215. ISSN: 1432-1378. DOI: [10.1007/s00145-002-0120-1](#).
- [Bel+22] M. Bellare et al. “Better than advertised security for non-interactive threshold signatures”. In: *Annual International Cryptology Conference (Crypto’22)*. Springer. 2022, pp. 517–550.
- [Ben+21] F. Benhamouda et al. “On the (in)security of ROS”. In: *Advances in Cryptology – EUROCRYPT 2021*. Cham: Springer International Publishing, 2021, pp. 33–53. ISBN: 978-3-030-77870-5.
- [Bol02] A. Boldyreva. “Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme”. In: *Public Key Cryptography — PKC 2003*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 31–46. ISBN: 978-3-540-36288-3.
- [CTZ24] R. Chairattana-Apirom, S. Tessaro, and C. Zhu. “Pairing-Free Blind Signatures from CDH Assumptions”. In: *Advances in Cryptology – CRYPTO 2024*. Cham: Springer Nature Switzerland, 2024, pp. 174–209. ISBN: 978-3-031-68376-3.
- [Cha83] D. Chaum. “Blind Signatures for Untraceable Payments”. In: *Advances in Cryptology*. Boston, MA: Springer US, 1983, pp. 199–203. ISBN: 978-1-4757-0602-4.

- [Cor00] J. Coron. “On the Exact Security of Full Domain Hash”. In: *Advances in Cryptology — CRYPTO 2000*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 229–235. ISBN: 978-3-540-44598-2.
- [DJW23] F. Denis, F. Jacobs, and C. Wood. *RSA Blind Signatures*. RFC 9474. Oct. 2023. DOI: [10.17487/RFC9474](https://doi.org/10.17487/RFC9474).
- [FW24] G. Fuchsbauer and M. Wolf. “Concurrently Secure Blind Schnorr Signatures”. In: *Advances in Cryptology – EUROCRYPT 2024*. Cham: Springer Nature Switzerland, 2024, pp. 124–160. ISBN: 978-3-031-58723-8.
- [FOO93] A. Fujioka, T. Okamoto, and K. Ohta. “A practical secret voting scheme for large scale elections”. In: *Advances in Cryptology — AUSCRYPT ’92*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 244–251. ISBN: 978-3-540-47976-5.
- [GG09] D. Galindo and F. Garcia. “A Schnorr-Like Lightweight Identity-Based Signature Scheme”. In: *Progress in Cryptology – AFRICACRYPT 2009*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 135–148. ISBN: 978-3-642-02384-2.
- [HLW23] L. Hanzlik, J. Loss, and B. Wagner. “Rai-Choo! Evolving Blind Signatures to the Next Level”. In: *Advances in Cryptology – EUROCRYPT 2023*. Cham: Springer Nature Switzerland, 2023, pp. 753–783. ISBN: 978-3-031-30589-4.
- [HKL19] E. Hauck, E. Kiltz, and J. Loss. “A Modular Treatment of Blind Signatures from Identification Schemes”. In: *Advances in Cryptology – EUROCRYPT 2019*. Cham: Springer International Publishing, 2019, pp. 345–375. ISBN: 978-3-030-17659-4.
- [Hau+20] E. Hauck et al. “Lattice-Based Blind Signatures, Revisited”. In: *Advances in Cryptology – CRYPTO 2020*. Cham: Springer International Publishing, 2020, pp. 500–529. ISBN: 978-3-030-56880-1.
- [Kat+24] S. Katsumata et al. “CSI-Otter: isogeny-based (partially) blind signatures from the class group action with a twist”. In: *Designs, Codes and Cryptography* 92.11 (Nov. 2024), pp. 3587–3643. ISSN: 1573-7586. DOI: [10.1007/s10623-024-01441-7](https://doi.org/10.1007/s10623-024-01441-7).
- [KLR21] J. Katz, J. Loss, and M. Rosenberg. “Boosting the Security of Blind Signature Schemes”. In: *Advances in Cryptology – ASIACRYPT 2021*. Cham: Springer International Publishing, 2021, pp. 468–492. ISBN: 978-3-030-92068-5.
- [Lys23] A. Lysyanskaya. “Security Analysis of RSA-BSSA”. In: *Public-Key Cryptography – PKC 2023*. Cham: Springer Nature Switzerland, 2023, pp. 251–280. ISBN: 978-3-031-31368-4.
- [QCY21] X. Qin, C. Cai, and T. Yuen. “One-More Unforgeability of Blind ECDSA”. In: *Computer Security – ESORICS 2021*. Cham: Springer International Publishing, 2021, pp. 313–331. ISBN: 978-3-030-88428-4.
- [Ran23] K. Randall. *NIST Special Publication (SP) 800-186, Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve*

- Domain Parameters* — *csrc.nist.gov*. <https://csrc.nist.gov/pubs/sp/800/186/final>. [Accessed 05-02-2025]. Apr. 2023.
- [Sch90] C. Schnorr. “Efficient Identification and Signatures for Smart Cards”. In: *Advances in Cryptology — CRYPTO’ 89 Proceedings*. New York, NY: Springer New York, 1990, pp. 239–252. ISBN: 978-0-387-34805-6.
- [Sch01] C. Schnorr. “Security of Blind Discrete Log Signatures against Interactive Attacks”. In: *Information and Communications Security*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–12. ISBN: 978-3-540-45600-1.
- [Wag02] D. Wagner. “A Generalized Birthday Problem”. In: *Advances in Cryptology — CRYPTO 2002*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 288–304. ISBN: 978-3-540-45708-4.
- [WNR20] P. Wuille, J. Nick, and T. Ruffing. *Schnorr Signatures for secp256k1*. Bitcoin improvement proposals. 2020. URL: <https://bips.xyz/340>.
- [WNT20] P. Wuille, J. Nick, and A. Towns. *Taproot: SegWit version 1 spending rules*. Bitcoin improvement proposals. 2020. URL: <https://bips.xyz/341>.

## A Attack on BZ

```
1 import hashlib
2 import time
3
4 # =====
5 # == PUBLIC PARAMETERS ==
6 # =====
7
8 #secp256k1
9 Zq = GF(
10     0xffffffffffffffffffffffffffffffffffffffffffffffffffffffffffc2f
11 )
12 E = EllipticCurve(Zq, [0, 7])
13 G = E.lift_x(
14     0x79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798
15 )
16 p = G.order()
17 Zp = GF(p)
18
19 # =====
20 # == SERVER SIDE CODE ==
21 # =====
22
23 class Server:
24     def __init__(self):
25         self._x = Zp.random_element()
26         self._X = G * self._x
27
```



```

28         self._sess = []
29
30     def get_pubkey(self):
31         return self._X
32
33     def commit(self):    # Returns (U, V, sess_id)
34         u, v = [Zp.random_element() for _ in range(2)]
35         self._sess.append([u, v, True])
36         return G * u, G * v, len(self._sess) - 1
37
38     def sign(self, sess_id, c, d):    # Returns w
39         assert self._sess[sess_id][2], f'Session {i} is not open'
40         self._sess[sess_id][2] = False
41
42         z = self._sess[sess_id][0] - c*self._x
43         return self._sess[sess_id][1] - d*z
44
45     def total_sessions(self):
46         return len(self._sess)
47
48
49     # =====
50     # === COMMON CODE (RANDOM ORACLES AND VERIFIER) ===
51     # =====
52
53     def oracle_H(U, m):
54         to_hash = str(G.xy()[0]) + str(U.xy()[0]) + m
55         hash = hashlib.sha512(to_hash.encode()).digest()
56         return Zp(int.from_bytes(hash, 'big'))
57
58     def oracle_G(V):
59         to_hash = str(G.xy()[0]) + str(V.xy()[0])
60         hash = hashlib.sha512(to_hash.encode()).digest()
61         return Zp(int.from_bytes(hash, 'big'))
62
63     def verify(pubkey, message, signature):
64         U, d, w = signature
65         c = oracle_H(U, message)
66         H = U - c * pubkey
67         V = G * w + H * d
68
69         assert d == oracle_G(V), f"verification equation fails on {message}"
70         return True
71
72
73     # =====
74     # === ATTACKER SIDE CODE ===
75     # =====
76
77     def inner_product(coefficients, values):

```

```

78     return sum(a*b for a, b in zip(coefficients, values))
79
80
81 start_time = time.time()
82
83 server = Server()
84 X = server.get_pubkey()
85
86 # open 'ell' sessions
87 ell = p.bit_length()
88 messages = [f"message{i}" for i in range(ell)] + ["forged message"]
89
90 # generate commitments
91 commits = [server.commit() for _ in range(ell)]
92 U = [comm[0] for comm in commits]
93 V = [comm[1] for comm in commits]
94
95 # generate blind parameters and challenges
96 alpha = [Zp.random_element() for i in range(ell)]
97 beta = [[Zp.random_element() for b in [0,1]] for i in range(ell)]
98 gama = [Zp.random_element() for i in range(ell)]
99 delta = [Zp.random_element() for i in range(ell)]
100
101 blind_U = [
102     [U[i]*alpha[i] + G*beta[i][b] for b in range(2)]
103     for i in range(ell)
104 ]
105 c = [
106     [oracle_H(blind_U[i][b], messages[i]) for b in range(2)]
107     for i in range(ell)
108 ]
109 all_blind_c = [
110     [c[i][b] / alpha[i] for b in [0,1]]
111     for i in range(ell)
112 ]
113
114 blind_V = [V[i]*alpha[i]*gama[i] + G*delta[i] for i in range(ell)]
115 d = [oracle_G(blind_V[i]) for i in range(ell)]
116 blind_d = [d[i] / gama[i] for i in range(ell)]
117
118 # generate linear combination and parameters of forged signature
119
120 #  $P(ep[i][b]) = 2^i * b$ 
121 ep = [
122     [all_blind_c[i][b] * blind_d[i] for b in [0, 1]]
123     for i in range(ell)
124 ]
125 p0 = -sum(Zp(2)^i * ep[i][0]/(ep[i][1] - ep[i][0]) for i in range(ell))
126 P = ([p0] + [Zp(2)^i / (ep[i][1] - ep[i][0]) for i in range(ell)])
127

```

```

128 targV = inner_product(P[1:], V)
129 targd = oracle_G(targV)
130 targU = inner_product(P[1:], [blind_d[i]*U[i] for i in range(ell)])
131 targU = (targd^-1) * targU
132 targc = oracle_H(targU, messages[-1])
133
134 # calculate the binary decomposition and send blind cs accordingly
135
136 y_to_dec = targc*targd + P[0]
137 bits = [int(b) for b in bin(y_to_dec)[2:].rjust(ell, '0')][::-1]
138 blind_c = [all_blind_c[i][b] for (i, b) in enumerate(bits)]
139
140 # receive the responses
141 w = [server.sign(i, blind_c[i], blind_d[i]) for i in range(ell)]
142
143 # generate the signatures
144
145 # normal signatures first
146 signatures = [
147     (blind_U[i][bits[i]], d[i],
148      w[i]*alpha[i]*gama[i] - d[i]*beta[i][bits[i]] + delta[i])
149     for i in range(ell)
150 ]
151
152 # forged signature
153 signatures += [(targU, targd, inner_product(P[1:], w))]
154
155 print('Are all signatures valid?', all(
156     [verify(X, messages[i], signatures[i]) for i in range(ell+1)]
157 ))
158
159 assert len(messages) == ell + 1
160 assert len(messages) == len(signatures)
161
162 print(f'Number of messages: {len(messages)}')
163 print(f'Number of open and closed sessions: {server.total_sessions()}')
164
165 total_time = time.time() - start_time
166 print(f'Total time to run: {total_time:.2f}s')

```

Listing 1.1: First attack implementation using Sage.

## B Attack on $m$ -BZ

```

1 import hashlib
2 import time
3
4 # =====

```

```

5 # === PUBLIC PARAMETERS ===
6 # =====
7
8 #secp256k1
9 Zq = GF(
10     0xfffffffffffffffffffffffffffffffffffffffffffffffffeffffc2f
11 )
12 E = EllipticCurve(Zq, [0, 7])
13 G = E.lift_x(
14     0x79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798
15 )
16 p = G.order()
17 Zp = GF(p)
18
19 # =====
20 # === SERVER SIDE CODE ===
21 # =====
22
23 class Server:
24     def __init__(self):
25         self._x = Zp.random_element()
26         self._X = G * self._x
27
28         self._sess = []
29
30     def get_pubkey(self):
31         return self._X
32
33     def commit(self): # Returns (U, V, sess_id)
34         u, v = [Zp.random_element() for _ in range(2)]
35         self._sess.append([u, v, True])
36         return G * u, G * v, len(self._sess) - 1
37
38     def sign(self, sess_id, c, d): # Returns w
39         assert self._sess[sess_id][2], f'Session {i} is not open'
40         self._sess[sess_id][2] = False
41
42         z = self._sess[sess_id][0] - c*self._x
43         return self._sess[sess_id][1] - d*z
44
45     def total_sessions(self):
46         return len(self._sess)
47
48
49 # =====
50 # === COMMON CODE (RANDOM ORACLES AND VERIFIER) ===
51 # =====
52
53 def oracle_H(U, V, m):
54     to_hash_c = str(G.xy()[0]) + str(U.xy()[0]) + str(V.xy()[0]) + m

```

```

55     to_hash_d = str(G.xy()[0]) + str(V.xy()[0]) + str(U.xy()[0]) + m
56     hash_c = hashlib.sha512(to_hash_c.encode()).digest()
57     hash_d = hashlib.sha512(to_hash_d.encode()).digest()
58
59     c = Zp(int.from_bytes(hash_c, 'big'))
60     d = Zp(int.from_bytes(hash_d, 'big'))
61     assert c != d
62     return c, d
63
64 def verify(pubkey, message, signature):
65     U, V, w = signature
66     c, d = oracle_H(U, V, message)
67     H = U - c * pubkey
68     V2 = G * w + H * d
69
70     return V2 == V
71
72 # =====
73 # === ATTACKER SIDE CODE ===
74 # =====
75
76 def inner_product(coefficients, values):
77     return sum(a*b for a, b in zip(coefficients, values))
78
79 start_time = time.time()
80
81 server = Server()
82 X = server.get_pubkey()
83
84 # open 'ell' sessions
85 ell = p.bit_length()
86 messages = [f"message{i}" for i in range(ell)] + ["forged message"]
87
88 # generate commitments
89 commits = [server.commit() for _ in range(ell)]
90 U = [comm[0] for comm in commits]
91 V = [comm[1] for comm in commits]
92
93 # generate random blind d's
94 blind_d = [Zp.random_element() for i in range(ell)]
95
96 # calculate V's of normal signatures
97 blind_V = [V[i] - U[i]*blind_d[i] for i in range(ell)]
98
99 # calculate U's of normal signatures
100 blind_olog = [[Zp.random_element() for b in [0,1]] for i in range(ell)]
101 blind_U = [[u * G for u in par] for par in blind_olog]
102
103 # generate challenges c's and d's

```

```

105 par_cd = [
106     [oracle_H(blind_U[i][b], blind_V[i], messages[i]) for b in [0,1]]
107     for i in range(ell)
108 ]
109
110 # generate blind c's
111 blind_c = [
112     [par_cd[i][b][0] * par_cd[i][b][1] / blind_d[i] for b in [0,1]]
113     for i in range(ell)
114 ]
115
116 # generate linear combination and parameters of forged signature
117
118 #  $P(ep[i][b]) = 2^i * b$ 
119 ep = [[blind_c[i][b] * blind_d[i] for b in [0, 1]] for i in range(ell)]
120 p0 = -sum([Zp(2)^i * ep[i][0] / (ep[i][1] - ep[i][0]) for i in range(ell)])
121 P = ([p0] + [Zp(2)^i / (ep[i][1] - ep[i][0]) for i in range(ell)])
122
123 prodV = inner_product(P[1:], V)
124 prodU = inner_product(P[1:], [blind_d[i] * U[i] for i in range(ell)])
125
126 targV = prodV - prodU
127 targulog = Zp.random_element()
128 targU = targulog * G
129
130 targc, targd = oracle_H(targU, targV, messages[-1])
131
132 # calculate the binary decomposition and send blind cs accordingly
133
134 y_to_dec = targc*targd + P[0]
135 bits = [int(b) for b in bin(y_to_dec)[2:].rjust(ell, '0')][::-1]
136 send_blind_c = [blind_c[i][b] for (i, b) in enumerate(bits)]
137
138 # receive the responses
139 w = [server.sign(i, send_blind_c[i], blind_d[i]) for i in range(ell)]
140
141 # generate the signatures
142
143 # normal signatures first
144 signatures = [
145     (blind_U[i][bits[i]], blind_V[i],
146      w[i] - par_cd[i][bits[i]][1] * blind_ulog[i][bits[i]])
147     for i in range(ell)
148 ]
149
150 # forged signature
151 signatures += [(targU, targV, inner_product(P[1:], w) - targd * targulog)
152               ↪ ]
153
154 print('Are all signatures valid?', all(

```

```

154     [verify(X, messages[i], signatures[i]) for i in range(ell+1)]
155 ))
156
157 assert len(messages) == ell + 1
158 assert len(messages) == len(signatures)
159
160 print(f'Number of messages: {len(messages)}')
161 print(f'Number of open and closed sessions: {server.total_sessions()}')
162
163 total_time = time.time() - start_time
164 print(f'Total time to run: {total_time:.2f}s')

```

Listing 1.2: Second attack implementation using Sage.